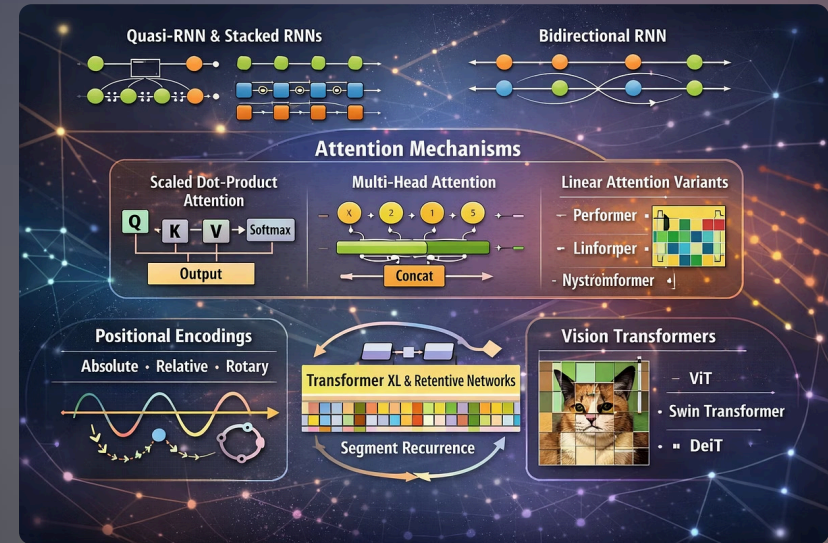


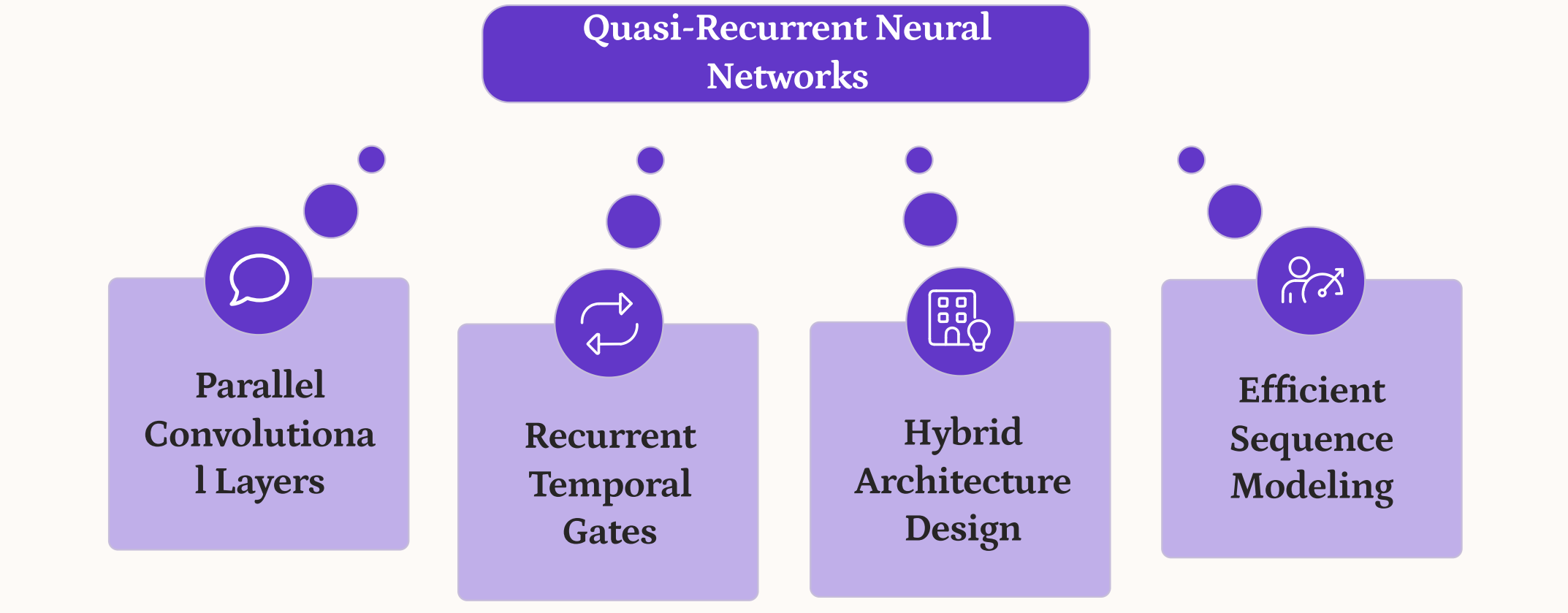
Module II:

Optimization and Efficient Training



Quasi-Recurrent Neural Network

Quasi-Recurrent Neural Networks (QRNNs) are a hybrid neural architecture designed to combine the parallelizable nature of Convolutional Neural Networks (CNNs) with the temporal dependency modeling of Recurrent Neural Networks (RNNs).



Main Goal of QRNN

Instead of doing full recurrence like LSTM:

- QRNN computes gates with convolution
- Only memory update is sequential

Architecture of QRNN

QRNN has 3 main components:

- Candidate values Z_t
- Forget gate F_t
- Output gate O_t

All computed using convolution:

$Z, F, O = \text{Conv1D}(X)$

Step-by-Step QRNN Working

Assume input sequence:

i

$X = [x_1, x_2, x_3, \dots, x_T]$

Step 1: Convolution Layer (Parallel)

QRNN applies convolution filters over the sequence.

1. Candidate State Z_t

i

$Z_t = \tanh(W_z * X_t)$

Meaning:

- This is the new information the network wants to add.

2. Forget Gate : F_t

$F_t = \text{sigmoid}(W_f * X_t)$

Meaning:

- Controls how much old memory to keep.

$F_t = 1 \rightarrow$ keep everything

$F_t = 0 \rightarrow$ forget everything

3. Output Gate : O_t

i

$O_t = \text{sigmoid}(W_o * X_t)$

Meaning:

- Controls how much memory should become output.

Step 2: Minimal Recurrence (Memory Update)

Now QRNN keeps memory using:

i

$c_t = F_t \odot c_{t-1} + (1 - F_t) \odot Z_t$

This is the only sequential part.

Breakdown:

Part 1: Keep old memory

i

$F_t \odot c_{t-1}$

Example:

If forget gate is high:

i

$F_t = 0.9$

Then:

- Keep 90% of previous memory

Part 2: Add new information

i

$(1 - F_t) \odot Z_t$

If:

i

$F_t = 0.2$

Then:

- Add 80% new info

Step 3: Output Hidden State

Final output:

i

$h_t = O_t \odot c_t$

Meaning:

- Output gate decides how much memory to show.

Example:

If:

i

$O_t = 0.5, c_t = 8.2$

Then:

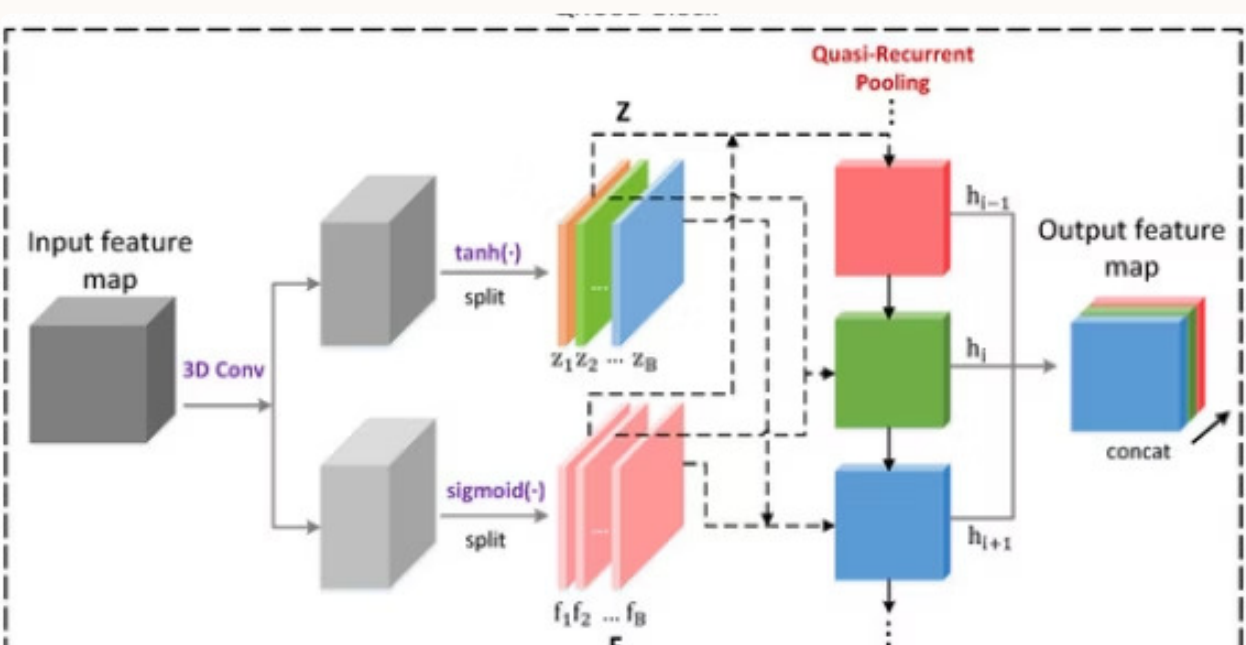
i

$h_t = 0.5(8.2) = 4.1$

So half memory is passed forward.

Full QRNN Flow Summary

Component	Role
Z_t	New candidate information
F_t	Forget gate (keep old memory?)
O_t	Output gate (how much to output?)
c_t	Memory cell
h_t	Final hidden state



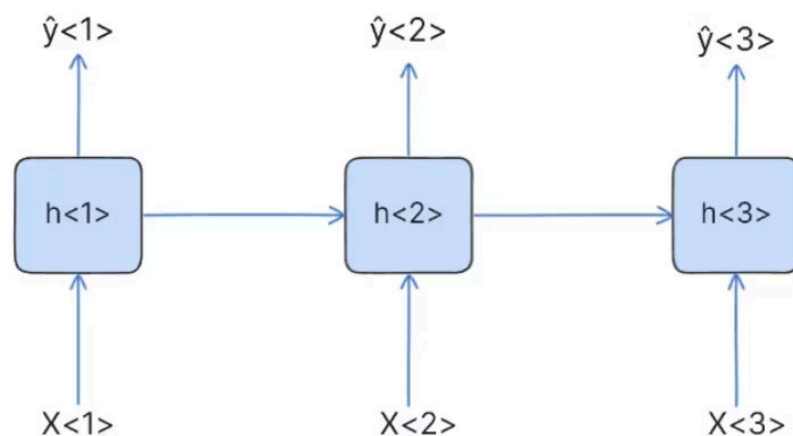
Bidirectional RNN

- A Bidirectional Recurrent Neural Network (BRNN) is a type of Recurrent Neural Network (RNN) that is designed to improve the performance of traditional RNNs by processing data in both forward and backward directions
- This architecture allows the network to have information from both past and future contexts, which can be particularly useful for tasks where context from both directions is crucial, such as language processing, speech recognition, and time-series analysis.

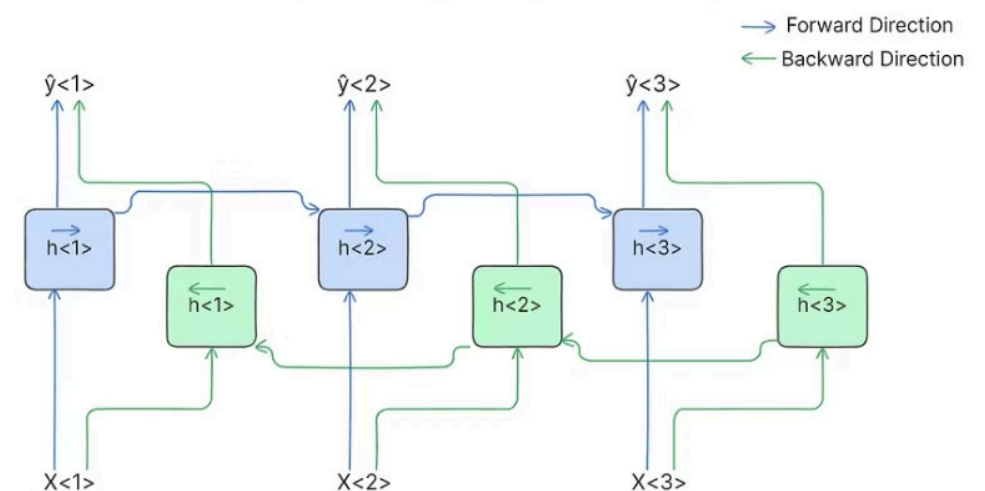
How BRNN Works

- A standard RNN processes input data in a sequence, maintaining a hidden state that gets updated at each step based on the current input and the previous hidden state. However, a standard RNN only uses past context, which can be a limitation for certain tasks
- A BRNN addresses this by having two separate hidden states: one that processes the sequence from start to end (forward direction) and another that processes it from end to start (backward direction). The outputs from these two hidden states are then combined (usually concatenated) to form the final output

Simple RNN Architecture



Bidirectional RNN Architecture

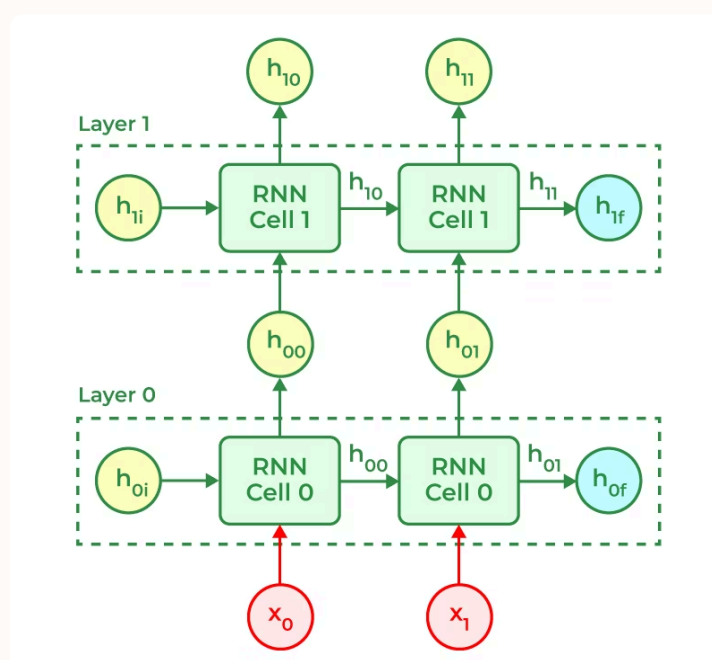


Stacked Recurrent Neural Networks

What are Stacked RNNs

A single-layered RNN model has only a hidden layer which is liable to process sequential data. But Stacked RNN is a special kind of model that has multiple RNN layers one on each layer. This creates a '**Stack**'. Each layer of this stack processes the input sequence.

- When an Input is passed to Layer 1:
 - The input (x_t) passes through the RNN layer 1. There, the hidden state gets updated as:
$$h_t = \sigma(w_x x_t + w_h h_{t-1} + b_h)$$
where
 - h_t = present Hidden state
 - h_{t-1} = previous hidden state
 - x_t = input to the RNN layer
 - w_x = weights associated with the input
 - w_h = weights associated with the hidden layer
 - b_h = bias associated with RNN layer
 - σ = activation function
 - What happens in the hidden state is that, using the information or knowledge it retained in the previous time step, the hidden state updates itself.
 - The present hidden state is used in getting the output of the hidden layer, using an appropriate activation function.
 - $y = W h_t + b_y$ where,
 - W = Weights assigned to the layer
 - h_t = hidden state
 - b_y = bias associated with output layer
- For the second layer, the output of first RNN layer is fed into it, which goes through the same process again.



Stacked RNN architecture

This feature of Stacked RNNs enables to capture of both short-term and long-term patterns. For that reason, Stacked RNNs can learn and remember information patterns in longer sequences and at the same it can analyze their current state's information with just learned previous state's information. The more layers you add to your model, the stacked network will be able to capture more complex patterns present in the sequential data. If your data is nested and has different types of complex patterns then Stacked RNNs will be a better model as its each layer can learn different abstractions present in your data.

What is an attention mechanism?

An attention mechanism is a [machine learning](#) technique that directs [deep learning](#) models to prioritize (or *attend to*) the most relevant parts of input data. Innovation in attention mechanisms enabled the transformer architecture that yielded the modern [large language models \(LLMs\)](#) that power popular applications like ChatGPT.

As their name suggests, attention mechanisms are inspired by the ability of humans (and other animals) to selectively pay more attention to salient details and ignore details that are less important in the moment. Having access to *all* information but focusing on only the *most relevant* information helps to ensure that no meaningful details are lost while enabling efficient use of limited memory and time.

Kernel Smoothing

In deep learning, **kernel smoothing has evolved from a classical statistical technique for noise reduction into a fundamental theoretical and architectural concept**. While traditionally used as a preprocessing step, it is now recognized as a core mechanism behind the success of modern architectures like **Transformers** and **ResNets**.

Attention as Kernel Smoother

Attention mechanisms in Transformers can be interpreted as **a form of non-parametric kernel smoothing, where the output is a weighted average of input values, with weights determined by a similarity kernel between queries and keys**. This perspective reframes the attention mechanism as a dynamic, data-dependent smoothing operation that enhances local information.

Key Connections between Attention and Kernel Smoothing:

- **Weighted Average:** Similar to traditional kernel smoothing, attention produces a weighted average of input features, where the weights determine how much information is passed from neighboring tokens.
- **Dynamic Kernel:** The "kernel" in attention is non-stationary; it is computed on-the-fly based on the query (Q) and key (K) pairs, allowing the model to adapt to different input sequences.
- **Softmax Normalization:** The attention score, usually calculated via, acts as the kernel function that controls the, intensity of smoothing between pairs of tokens.
- **Multi-Head Perspective:** Multi-head attention can be interpreted as using multiple, specialized kernels, which allows the model to capture different, types of relationships simultaneously.

Scaled Dot-Product Attention and Multi-Head Attention

Scaled Dot-Product Attention

This is the basic attention mechanism used in Transformers.

Formula of Scaled Dot-Product Attention

i

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

What is Multi-Head Attention?

Multi-head attention extends self-attention by splitting the input into multiple heads, enabling the model to capture diverse relationships and patterns.

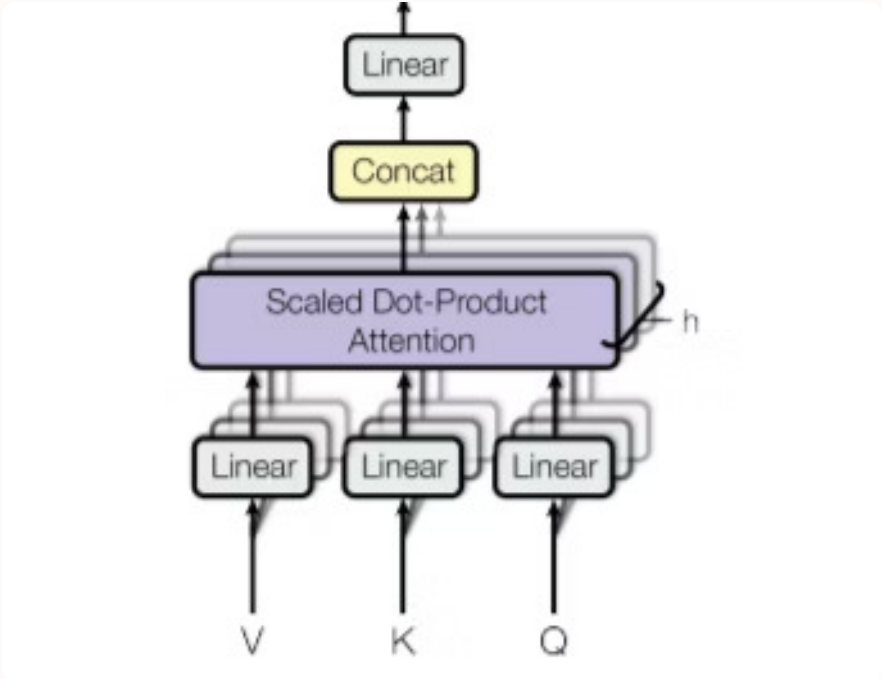
Instead of using a single set of Q, K, V matrices, the input embeddings are projected into multiple sets (heads), each with its own Q, K, V :

1. **Linear Transformation:** The input X is projected into multiple smaller-dimensional subspaces using different weight matrices.

$$Q_i = XW_i^Q, K_i = XW_i^K, V_i = XW_i^V$$

where i denotes the head index.

1. **Independent Attention Computation:** Each head independently computes its own self-attention using the scaled dot-product formula.
2. **Concatenation:** The outputs from all heads are concatenated.
3. **Final Linear Transformation:** A final weight matrix is applied to transform the concatenated output into the desired dimension.



Multi-Head Attention

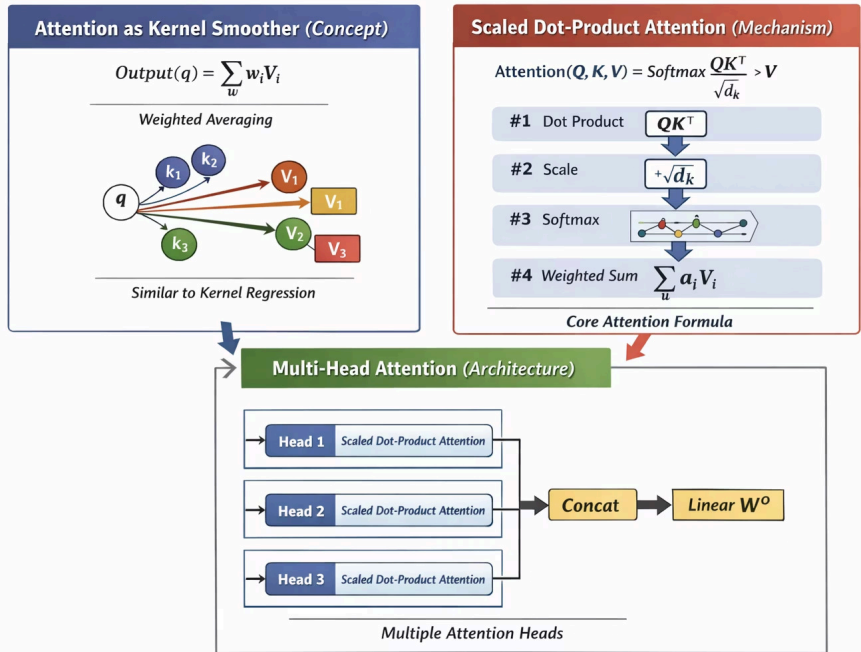
Mathematically, multi-head attention is expressed as:

$$MultiHead(Q, K, V) = Concat(head1, head2, \dots, headh)W^O$$

where:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

W^O is a final weight matrix to project the concatenated output back into the model's required dimensions.



Kernel smoother = *what attention resembles*

Scaled dot-product = *how attention is computed*

Multi-head = *how attention is made stronger*

Linear Attention Variants

Linear attention variants are a class of efficient attention mechanisms designed to overcome the quadratic ($O(N^2)$) complexity of standard softmax-based attention in Transformers, reducing it to linear ($O(N)$) complexity with respect to sequence length. They achieve this by using kernel functions, matrix-valued state accumulation, or low-rank approximations to avoid computing the full $N * N$ attention matrix.

1

Performer

Performer is a linear attention Transformer that approximates softmax attention using random feature mappings (FAVOR+), avoiding the explicit $n \times n$ attention matrix. This reduces attention complexity from $O(n^2)$ to $O(n)$, making Transformers efficient for very long sequences.

Formula: $\text{Att}(Q, K, V) \approx \phi(Q)\phi(K)TV$

2

Linformer

Linformer makes attention linear by assuming the attention matrix is low-rank and compressing keys and values using learned projections.

Formula: $K' = EK, V' = FV$

Attention = $\text{softmax}(QK'T)V'$

3

Nystromformer

Nyströmformer is a linear attention variant that approximates the full self-attention matrix using a small set of landmark tokens

Formula: $\text{Att}(Q, K, V) \approx A_{n,m} A_{m,m-1} A_{m,n} V$

Positional Encodings

Positional encoding **injects information about the order of tokens into Transformer models, which process input in parallel rather than sequentially**. It adds a unique vector (sinusoidal or learned) to input embeddings to represent each token's position, allowing the model to distinguish between different word orders

Absolute:

Absolute positional encoding assigns a **fixed position vector** to each token based on its position in the sequence.

Rotary:

- Encodes positions **using rotations in embedding space**
- Apply a **rotation matrix** to query and key embeddings

Relative:

Instead of absolute positions, encode **distance between tokens**.

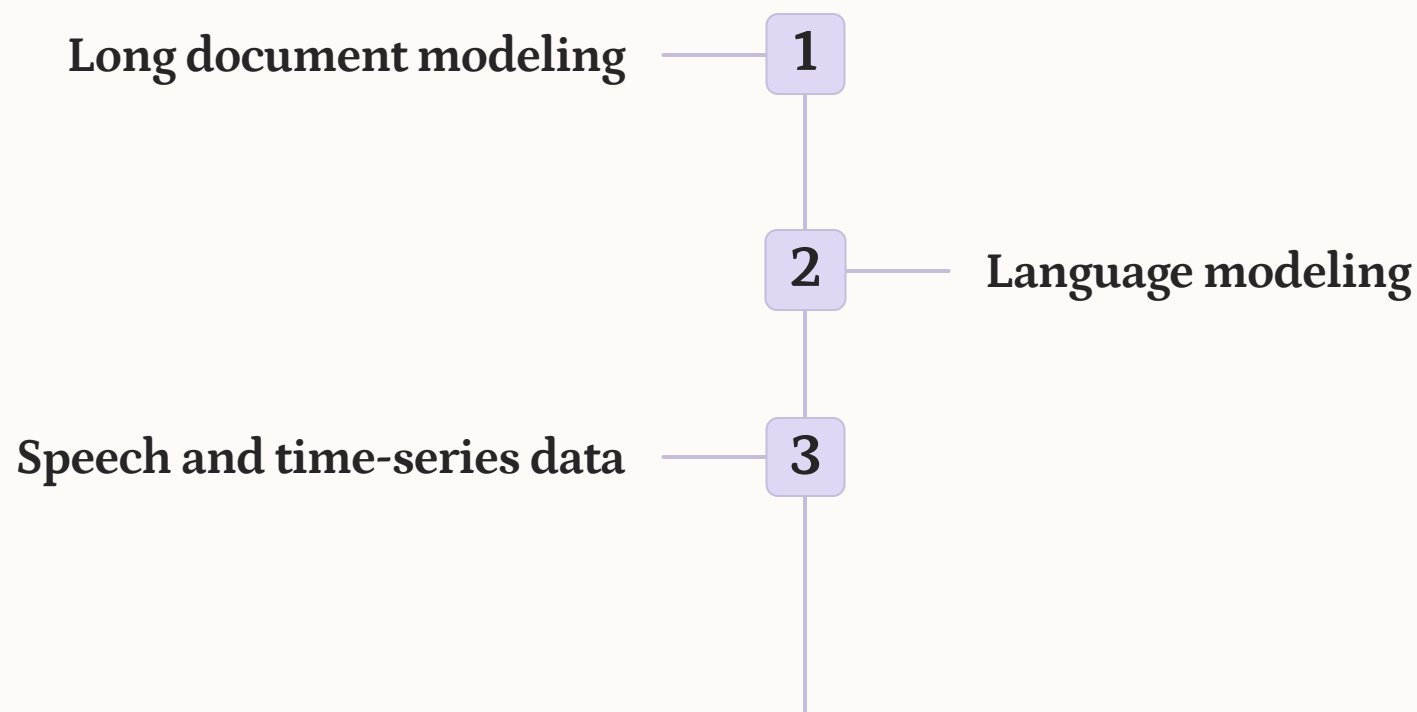
Attention with Linear-Biases:

- Don't modify embeddings directly
- Add a **linearly increasing bias** to attention scores

Transformer Extra-Long Segment Recurrence

- **Transformer-XL** is an extension of the standard Transformer designed to handle **extra-long sequences** beyond a fixed context window.
- Standard Transformers process inputs in independent segments, which causes them to **lose long-range dependencies** across segments.
- Transformer-XL introduces a **segment-level recurrence mechanism**, where hidden states from previous segments are reused as a **memory** for the next segment.
- This memory allows the model to maintain context across multiple segments, enabling learning of **long-term dependencies** in text and sequential data.
- It uses **relative positional encoding**, which helps preserve positional relationships even when information is carried across segments.

Transformer-XL improves performance on tasks requiring long context, such as:



Key Advantages:

- 1** Captures dependencies over much longer ranges
- 2** More efficient than attending over the full sequence
- 3** Enables context continuity across segments

Main Contribution:

Combines Transformer attention with recurrence-style memory to support long-context understanding.

Retentive Network with Gated Attention (RetNet)

- **Retentive Networks (RetNet)** are designed as an efficient alternative to Transformers for handling **very long sequences**.
- They address the major limitation of standard self-attention, which becomes slow and memory-heavy for long inputs.
- Instead of computing attention between every pair of tokens, RetNet uses a **retention mechanism** that maintains a compact memory of past information.
- A **gating mechanism** controls how much previous memory should be kept versus how much new information should be added.
- This gating helps the model focus on important long-term context while preventing irrelevant past information from dominating.

RetNet combines the strengths of:

- Transformers (strong contextual representations)
- RNNs (efficient sequential memory)

The model achieves **linear-time efficiency**, making it suitable for extremely long documents, long-context language models, and sequential data like audio or video.

Key Advantage

Long-range dependency modeling with much lower computation and memory cost compared to standard Transformers.

Comparative Analysis

Model	Main Mechanism	Context Handling	Long-Range Dependency	Efficiency	Key Limitation
RNN	Sequential hidden-state recurrence	Processes tokens one-by-one	Weak for very long sequences (vanishing gradients)	Linear but slow due to no parallelism	Cannot parallelize, limited long memory
Transformer	Full self-attention over tokens	Fixed-length context window	Strong within window, but limited beyond it	Expensive for long sequences	High computation and memory cost for long inputs
Transformer-XL	Attention + segment-level recurrence (memory)	Reuses past segment hidden states	Better long-term dependency modeling	More efficient than full long attention	Memory is limited to recent segments
Retentive Network (RetNet)	Retention-based memory + gating	Maintains compressed long-term memory	Excellent for extremely long contexts	Linear-time and memory efficient	Approximate interactions (not full attention)

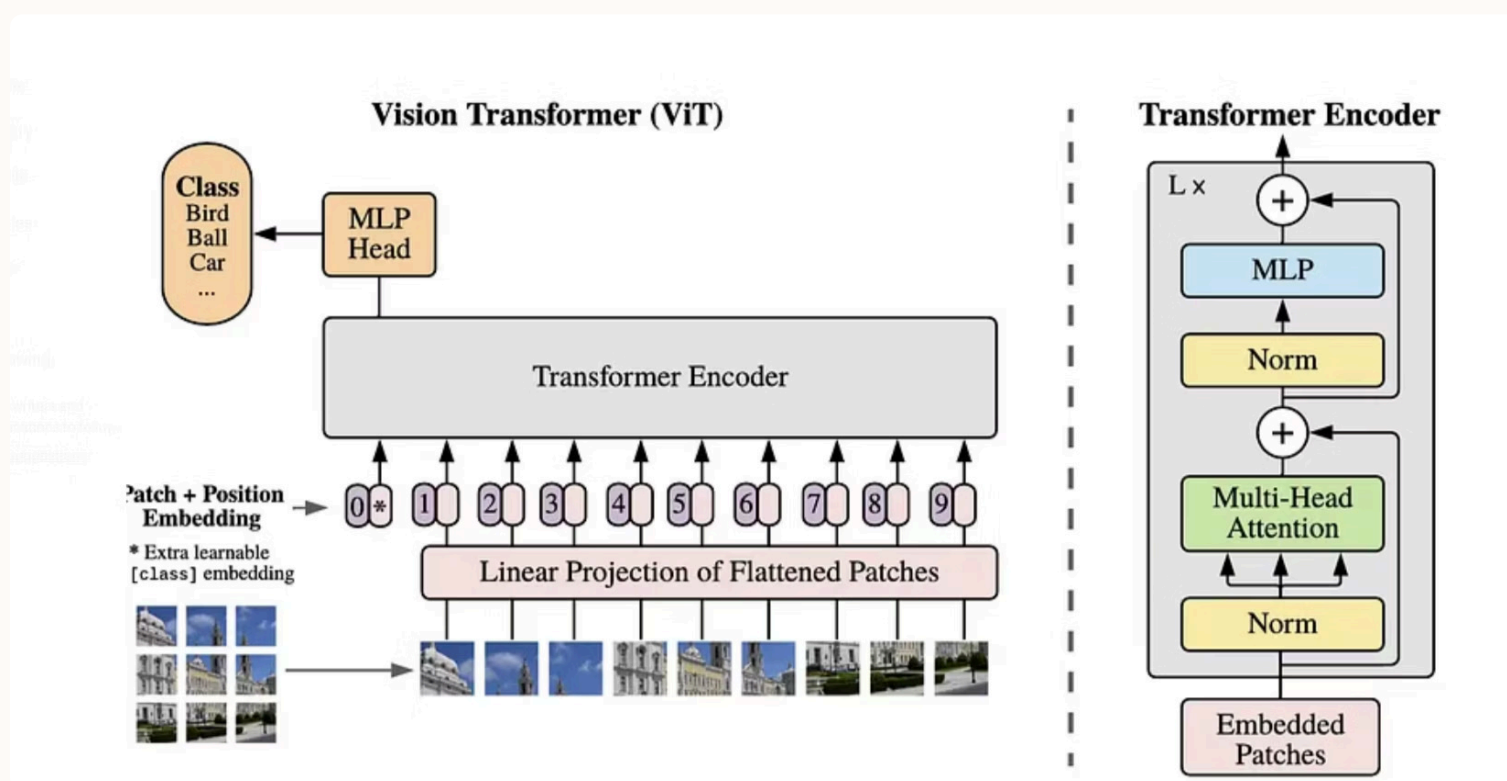
Vision Transformers

A Vision Transformer (ViT) is a **deep learning model that applies transformer architectures, originally designed for NLP, to computer vision tasks by treating image patches as tokens**

In traditional image processing tasks, Convolutional Neural Networks (CNNs) were the go-to method. They excelled at detecting patterns and features through a grid-like structure, making them great for images, but CNNs struggled to model **global relationships** between different parts of an image.

To address these limitations, **Transformers**, originally designed for text, were adapted for computer vision tasks. Unlike CNNs, Transformers can model long-range dependencies in data, which is essential for both NLP and image understanding. Transformers were designed for **sequential data** (like sentences). But images are structured as a grid of pixels. The main task was to "convert" an image into a format the transformer could process while leveraging the powerful **self-attention mechanism** of Transformers.

Architecture of ViT



Patchify the Image:

Split the image into smaller patches. Imagine an image of size 128x128 pixels. ViT splits this image into smaller patches, say 16x16 patches. For a 128x128 image, this would create **64 patches** ($128 \div 16 = 8$ patches along width, and 8 patches along height, making $8 \times 8 = 64$ patches).

Linear Projection:

Flatten each patch and project it into a vector (token). These patches are flattened into a vector, like turning a mini image into a **word** in a sentence.

Positional Encoding:

Just as words in a sentence need to be understood in order (the word "car" and "fast" have different meanings based on their order), patches in an image need their **position** to be understood. Positional encoding is added to the patch tokens to inform the Transformer of where each patch is located in the original image.

Transformer Encoder:

Process the sequence of image tokens through Transformer layers. Each patch, now represented as a token, passes through the **Transformer** encoder. The self-attention mechanism helps each patch learn about other patches in the image. For example, the patch showing the wheel of a car will learn information about the patch showing the car's body.

Classification Token:

Process the sequence of image tokens through Transformer layers. Each patch, now represented as a token, passes through the **Transformer** encoder. The self-attention mechanism helps each patch learn about other patches in the image. For example, the patch showing the wheel of a car will learn information about the patch showing the car's body.

Data-Efficient Image Transformer

A data-efficient image transformer is a **vision transformer model or transformer-based approach designed to achieve strong performance with relatively small amounts of labeled image data.**

Data-Efficient Image Transformer (DeiT) is a **vision model developed by Meta AI (formerly Facebook AI)** that enables pure Vision Transformers (ViT) to be trained effectively **on standard, mid-sized datasets like ImageNet-1K (1.2 million images)**

s based on the standard Vision Transformer architecture with an additional **distillation mechanism** to improve data efficiency.

1 Token-Based Distillation

DeiT introduces a **distillation token** alongside the standard **[CLS] token** and image patch tokens.

- The **[CLS] token** is trained using ground-truth labels.
- The **distillation token** is trained to mimic the predictions of a CNN teacher model (typically ResNet).
- All tokens interact through self-attention layers.
- DeiT uses **hard distillation**, where the student model learns from the teacher's top predicted class instead of full probability distributions.

This mechanism allows the transformer to inherit CNN inductive biases while maintaining the transformer architecture.

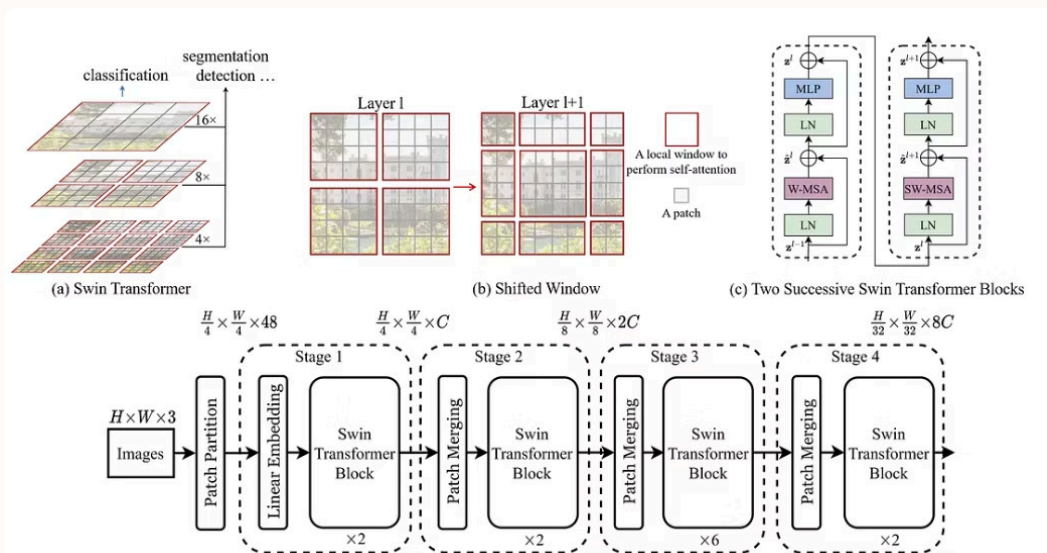
2 Strong Training Strategy

DeiT achieves data efficiency through advanced training techniques:

- **Data Augmentation:** RandAugment, Mixup, and CutMix are applied to increase training diversity.
- **Regularization:** Stochastic depth and repeated augmentation are used to prevent overfitting on the ImageNet-1K dataset (1.2M images).

Swin Transformer

Swin Transformer is a **hierarchical vision transformer that uses shifted windows to achieve efficient, linear-complexity image processing**, serving as a versatile, state-of-the-art backbone for computer vision tasks. It addresses the high-resolution, quadratic-complexity limitations of standard Transformers by calculating self-attention within local, non-overlapping windows that shift between layers, enabling cross-window connections.



Core Architecture Idea

1 Patch Partition

Image $\rightarrow$ divided into small patches (e.g., 4×4)

2 Window-Based Self-Attention

Instead of global attention:

- Attention is computed only inside small windows.
- Reduces computation from quadratic to linear complexity.

👉 This makes it scalable to high-resolution images.

3 Shifted Window Mechanism

In the next layer:

- Windows are shifted slightly.
- This allows cross-window information exchange.

So model:

- Keeps efficiency (local attention)
- But still captures global context over layers

4 Hierarchical Design

Unlike ViT, Swin:

- Reduces spatial resolution stage-by-stage
- Increases channel dimension
- Similar to CNN pyramid structure

This makes it suitable for:

- Object detection
- Segmentation
- Medical imaging

